

# A Deep Dive into POSIX Message Queues

This article throws some light on the advanced concepts of POSIX message queues. A basic knowledge of these queues will be needed to understand and appreciate the article better.



**P**OSIX message queues are a means of inter-process communication (IPC) based on message exchange. Two or more processes exchange data in the form of messages (as a whole, without any provision to exchange partial bytes of the message), and can prioritise these messages by allowing the sender to set the priority through a parameter accompanying the message. Programs intending to use POSIX message queues must include real-time library *librt*. POSIX message queues are available to Linux user space, but the messages are stored in kernel space in a virtual file system. One can view the created POSIX message queues in `/dev/mqueue/`. POSIX message queue support is an optional kernel component that is configured via `CONFIG_POSIX_MQUEUE` kernel configuration option, and is enabled by default. There are other implementations that may provide such a feature, but the details are likely to differ. Using POSIX message queues, data structures can also be shared between processes in the form of messages.

## Limitations of using POSIX message queues

POSIX message queues are message based inter-process communication (IPC), which means the entire message

is received at once by the reading process when it calls `mq_read()`. Reading part of a message or just a few bytes of a message is not possible. In addition, there is no way to check how many unread messages are present in the message queue.

It is only possible to find out how many bytes are available to be read from the message queue. This is the total size (in bytes) of unread messages, which can be more than one message. However, it is not possible to break down these bytes and find out how many messages these unread bytes in the queue comprise.

To find out how many unread bytes are in the message queue, a process can call the `mq_getattr()` API. The API `mq_getattr()` takes two parameters — input message queue descriptor of the intended queue and an output parameter, as shown below:

```
int mq_getattr(mqd_t mqdes, struct mq_attr *attr);
```

The output parameter of type `struct mq_attr` has a member element of type `mq_curmsgs`, which tells the number of bytes available in the message queue to be read.